

ExpenseEase - Mid-Semester Project Report

1. Cover page

Project: ExpenseEase

Group number: Pending team confirmation

Members and registration numbers: Shreeya Srivastava (20243267); Shraddha Sharma (20243266); Rudransh Pratap Singh (20243243); Sachit Jain (20243245)

Course and section: Software Engineering & Project Management (CSN15101); section pending confirmation

Supervisor/instructor: [Dr. Satarupa

Chakrabarti](<https://www.linkedin.com/in/dr-satarupa-chakrabarti-033221242/?isSelfProfile=false>)

2. Problem statement and motivation

Friends, roommates, and trip groups often use chat messages or spreadsheets to track shared spending. This causes forgotten IOUs, unclear payers, unequal splits, and circular debts. ExpenseEase provides a local web application to record group expenses, compute exact balances, and recommend a smaller set of repayments.

3. Project objectives

1. Record expenses with equal, exact, and percentage split rules.
2. Preserve monetary accuracy using integer paise.
3. Compute net balances and simplified debt transfers.
4. Protect group data with authentication and authorization.
5. Maintain an explainable, testable codebase.

4. Scope

Included scope is authentication, group membership, expenses, balances, settlements, activity, and dashboard summaries. The current implementation provides a live registration-to-group-to-expense workflow backed by PostgreSQL. Payment processing, OCR, multi-currency support, social login, notifications, native apps, and recurring expenses remain out of scope. The application uses INR, Dockerized PostgreSQL, and manually recorded settlements.

5. Stakeholders

Stakeholder	Interest	Influence
Group member	Accurate shared costs	High
Group admin	Membership and data oversight	High
Course evaluator	Evidence of engineering practice	High
Development team	Deliverable quality and learning	High
Future maintainer	Clear design and tests	Medium

6. Requirements specification

Functional requirements cover registration, group creation, member addition, expense CRUD, split validation, balance calculation, debt simplification, settlement recording, activity, and dashboard summaries. Non-functional requirements include bcrypt password hashing, JWT-based authentication, validated input, Dockerized PostgreSQL, and balance invariants.

7. Requirements model

The detailed use-case diagram, six use cases, and eighteen user stories are in the requirements document. The core acceptance condition remains that submitted expense shares must total exactly to the stored paise amount and that only group members can access a group.

8. System design

The browser SPA calls an Express REST API. Routes delegate to controllers and services; Prisma accesses PostgreSQL. The debt simplifier is a pure service, separated from database access. Balances are computed from expenses and settlements rather than stored, preventing stale records.

9. Technology stack and justification

Layer	Choice	Why
Frontend	React/Vite/Axios	Fast local SPA and API integration
Backend	Node.js/Express	Small REST service with clear routes
Validation	Zod	Declarative input contracts
Database	PostgreSQL/Prisma	Dockerized relational persistence
Security	bcrypt/JWT/helmet	Password hashing and protected routes
Tests	Vitest/Supertest	Fast unit and API-oriented testing

10. Implementation progress

Verified backend capabilities in the current repository include authentication, group membership and authorization, expense CRUD, deterministic split validation, balance calculation, debt simplification, settlement support, activity, dashboard summaries, and service-level tests. The current frontend implements a live registration-to-group-to-expense-to-balance workflow.

The frontend supports equal, exact, and percentage split input flows, expense editing/deletion, settlement recording, balance views, activity, and dashboard summaries.

11. Testing performed

On 30 September 2026, npm test completed successfully with 157 server tests and 22 client tests passing (179 total). npm run lint and npm run build also completed successfully. The server tests run against the isolated expenseease_test PostgreSQL database.

12. Project-management status

The project uses an iterative and incremental SDLC: requirements and design establish a traceable baseline, then each working increment is implemented, integrated, tested, and reviewed before the next increment. The completed increments are authentication/group workflows, expense/split workflows, and balance/settlement/dashboard workflows. This approach provided working software early, controlled change through scoped increments, and used

automated tests and screenshots as validation evidence.

The project-management documents contain the live status and discovered risks. Git history confirms implemented backend services and frontend authentication, equal/exact/percentage splits, settlement, and group workflows. Each of the four members has an equal 25% contribution spanning frontend and backend work; the primary ownership record is in Task_Allocation.md and Contribution_Record.md.

13. Individual contribution table

Member	Tasks done	Modules / commits	Percentage
Shreeya Srivastava (20243267)	Group and membership integration	Group/group-data APIs, membership rules, group UI, tests, documentation review	25%
Shraddha Sharma (20243266)	Authentication requirements and integration	Auth API, validation, JWT, registration/login UI, tests, README review	25%
Rudransh Pratap Singh (20243243)	Expense and split integration	Expense APIs, split validation, expense/activity UI, tests, test-case review	25%
Sachit Jain (20243245)	Balance and settlement integration	Balance/debt/settlement APIs, dashboard/balance UI, Docker verification, evidence, demo preparation	25%

14. Challenges faced and solutions attempted

The project addressed deterministic rounding by allocating leftover paise in ascending user-id order. Simplification matches the largest creditor and debtor deterministically. Authorization is enforced through membership and creator/admin checks. Balance consistency is maintained by calculating from records rather than storing a cached balance. Prisma Client generation is included in the Docker build and setup workflow, and the authentication suite now passes.

15. Remaining-semester work

Capture screenshots and verified test evidence, complete truthful team/project-management details, record and publish the demo video, export required PDFs, improve accessibility and responsiveness, and prepare the final submission package.

16. References

1. Prisma Documentation, <https://www.prisma.io/docs>
2. Express Documentation, <https://expressjs.com/>
3. React Documentation, <https://react.dev/>
4. OWASP Top 10, <https://owasp.org/www-project-top-ten/>
5. Ian Sommerville, Software Engineering.
6. Course lecture material: Life Cycle Models; Incremental, Iterative and Prototyping Development Models; Requirements Validation, Traceability and Change Management; Software Design Principles; Software Testing.

Appendix A. AI-use disclosure

See AI_USAGE_DISCLOSURE.md. Students must complete the verification/modification column honestly and remain responsible for correctness, security, and originality.